ResourceDesk - Project Documentation

ResourceDesk is a role-based internal resource reservation and approval application for organizations. It helps employees discover shared resources, request bookings, and track decisions while giving department heads and administrators the controls needed to manage availability and access.

> **Project status:** Implemented as an internship assessment prototype. This document describes the intended product behavior and the implemented workflows; production deployment and environment-specific verification should be recorded separately.

1. Business Problem and Product Vision

Shared meeting rooms, equipment, and other organizational resources are often booked through informal messages or spreadsheets. This makes availability difficult to check, creates overlapping requests, and leaves employees uncertain about approval status.

Vision: Provide one reliable place to find organizational resources, request time slots, apply approval rules, and manage reservations with clear visibility for employees, department heads, and administrators.

Primary goals:

- Make resource discovery and booking straightforward.
- Prevent conflicting confirmed reservations.
- Support department-level ownership and approval.
- Give users visibility into their reservations and upcoming bookings.
- Allow administrators to manage resources and onboard employees securely.

2. Scope and Priorities

Priority	Included capabilities
Core	Authentication, role-based access, resource directory and details, reservation requests, availability/conflict handling, personal reservations
Core	Automatic confirmation when approval is not required; pending review when approval is required
Core	Department-head/administrator approval and rejection workflows
Management	Resource administration, dashboard summaries, employee invitations and account activation
Supporting	Demonstration data, responsive navigation, status feedback and validation

3. Actors, Access, and User Journeys

Actor	Main permissions
Employee	Sign in, browse permitted resources, create reservation requests, view/manage own reservations
Department Head	Employee capabilities plus review requests within assigned approval authority
Administrator	Organization-wide oversight, resource management, employee invitations, and administrative review

Employee journey: Sign in → browse/search resources → inspect resource details → choose date and time → submit reservation → see confirmed or pending status → track the booking in My Reservations.

Approval journey: Authorized reviewer signs in → opens Approvals → inspects a pending request → approves or rejects → the reservation status and review information are updated.

Employee onboarding journey: Administrator invites an employee and assigns a department/role → the system emails a time-limited activation link → employee sets their name and password → account becomes active → employee signs in.

Access is checked server-side; navigation visibility alone is not considered authorization.

4. Functional Requirements

ID	Requirement
FR-01	Authenticate active users and enforce Employee, Department Head, and Administrator roles.
FR-02	Display resources with relevant details, status, department ownership, and access scope.
FR-03	Let eligible users submit reservations with purpose, start time, and end time.
FR-04	Validate time ranges, resource eligibility, and reservation conflicts.
FR-05	Confirm eligible bookings directly or mark them pending when approval is required.
FR-06	Let users view their reservations and perform permitted cancellation actions.
FR-07	Let authorized reviewers approve or reject pending requests and record review details.
FR-08	Let administrators manage resource records and view employee information.
FR-09	Let administrators invite employees by email, department, and role.
FR-10	Let invited employees activate their accounts using a single-use, expiring link.
FR-11	Display role-appropriate dashboard summaries and upcoming bookings.

5. Non-Functional Requirements

- Security: Passwords are hashed; authentication and role checks protect server operations; activation tokens are stored as hashes rather than plaintext.
- Data integrity: PostgreSQL relationships and constraints protect records; a database exclusion constraint prevents overlapping confirmed reservations for the same resource.
- Usability: Clear statuses, form validation, responsive layout, and straightforward navigation.
- Maintainability: TypeScript, reusable React components, Prisma models, and separated server actions/services.
- Reliability: Reservation updates and account activation use database-backed operations; user-facing errors should be understandable.
- Configuration**:** Secrets and connection strings are provided through environment variables, not committed to source control.

6. Business Rules and Edge Cases

- Only ACTIVE accounts can sign in. Newly invited accounts remain INVITED until activation.
- Resource access depends on its status and access scope (ORGANIZATION or DEPARTMENT).
- A reservation must have a valid start/end time and must satisfy the application's availability checks.
- Resources with requiresApproval = true follow the pending-review workflow; eligible requests for other resources can be confirmed without review.
- Confirmed reservations for the same resource must not overlap. The database constraint is a final safeguard against concurrent conflicting confirmations.
- Reservation statuses are PENDING, CONFIRMED, REJECTED, and CANCELLED.
- Resource statuses are ACTIVE, MAINTENANCE, and INACTIVE.
- Rejection reasons and review timestamps are retained where applicable.
- An invitation link expires after 24 hours and cannot be reused after successful activation.
- Invalid, expired, already-used, or otherwise ineligible activation tokens must not activate an account.
- Unauthorized users must not gain access by manually entering protected URLs.

7. Data Model and ER Design

The application uses PostgreSQL (Neon) through Prisma ORM. The conceptual ER diagram is maintained separately as Entity Relationship Diagram.pdf.

Entity	Purpose and key fields
Department	Department identity (id, departmentCode, name), optional department head, creation timestamp
User	Identity (id, employeeId, name, email), hashed password, role, status, optional department, creation timestamp
Resource	Resource identity (id, resourceCode, name), category, location, description, capacity, status, access scope, approval requirement, optional owning department
Reservation	Booking reference, requesting user, resource, purpose, time interval, status, optional approver/reviewer, rejection reason, review and creation timestamps
AccountActivationToken	Hashed invitation token, owning user, expiry, optional use timestamp, creation timestamp

Key relationships: Department → Users (one-to-many); Department ↔ Head (optional one-to-one); Department → Resources (one-to-many, optional ownership); User → Reservations (one-to-many); Resource → Reservations (one-to-many); User → Approval/Review assignments (optional per reservation); User → Activation Tokens (one-to-many).

Primary keys use UUID identifiers. Codes, booking references, and relevant user identifiers have uniqueness constraints as defined in the Prisma schema.

8. Architecture and Project Structure

Technology stack: Next.js 16 (App Router), React 19, TypeScript, Tailwind CSS 4, Auth.js v5, Prisma 5, Neon PostgreSQL, bcrypt, Zod, and Resend for invitation email delivery.

Request flow: Browser UI → Next.js pages/components → authenticated server actions/services → Prisma → Neon PostgreSQL. Auth.js manages sessions and role information; Resend sends activation emails.

```
resourcedesk/
├── prisma/
│   ├── schema.prisma                # Database models and relationships
│   └── migrations/                  # Versioned schema changes
├── scripts/
│   └── demo-data.cjs                # Development demonstration data
├── src/
│   ├── app/
│   │   ├── (protected)/             # Authenticated application pages
│   │   │   ├── dashboard/
│   │   │   ├── resources/
│   │   │   ├── reservations/
│   │   │   └── admin/
│   │   ├── activate/                # Invitation activation
│   │   └── login/                   # Sign-in
│   ├── components/                  # UI, forms, layout and workflows
│   ├── server/
│   │   ├── actions/                 # Server-side mutations
│   │   └── services/                # Business/integration logic
│   └── auth.ts                      # Authentication configuration (at src/auth.ts)
└── package.json
```

9. UI/UX Design and Key Decisions

The interface uses a professional navy-and-neutral visual style, a consistent sidebar/header, readable status labels, and responsive cards, forms, and tables.

Page / feature	Design intent
Login / Account Activation	Focused authentication and first-time account setup
Dashboard	Role-relevant summaries and upcoming resource bookings
Resource Directory / Details	Easy discovery and clear resource information
Quick Reservation	Short booking flow with explicit date, time, and purpose
My Reservations	Personal booking history and status visibility
Approvals	Dedicated queue for authorized reviewers
Admin Resources	Centralized resource maintenance
Employee Management	Controlled employee invitation and account visibility

Important product decisions:

- Use role-based access instead of separate applications for each actor.
- Support both organization-wide and department-specific resources.
- Require approval only for resources configured to need it.
- Keep reservation state explicit rather than treating all requests as confirmed.
- Prevent overlapping confirmed bookings at the database level, not only in the UI.
- Use administrator invitations rather than unrestricted public registration.
- Keep onboarding links short-lived, single-use, and hashed in storage.

10. Development Setup

Prerequisites: Node.js, npm, a PostgreSQL/Neon database, and valid environment configuration.

```
npm install
npx prisma generate
npx prisma migrate dev
npm run dev
```

Open `http://localhost:3000`.

Environment configuration

```
DATABASE_URL="<neon-pooled-connection-string>"
DIRECT_URL="<neon-direct-connection-string>"
AUTH_SECRET="<strong-secret>"
RESEND_API_KEY="<resend-api-key>"
EMAIL_FROM="ResourceDesk <sender@verified-domain.example>"
APP_URL="http://localhost:3000"
```

For deployment, use the real HTTPS application URL as APP_URL, configure the authentication host settings as required by the hosting environment, and use a verified sender domain for unrestricted employee invitations. Never commit .env or real secrets.

Quality and production commands:

```
npx tsc --noEmit
npm run lint
npm run build
npx prisma migrate deploy
```

prisma migrate deploy is for applying committed migrations to the target deployment database. The deployment build should generate Prisma Client (for example, prisma generate && next build). Run the demo-data script only intentionally in a non-production environment.

11. Implementation Milestones

1. Foundation: Next.js application, styling, Prisma schema, database connectivity, authentication.
2. Access and resource discovery: Roles, protected layout, resource directory and details.
3. Reservation workflow: Booking creation, validation, status handling, and conflict prevention.
4. Approvals and administration: Department-head review, admin resource management, dashboards.
5. Experience improvements: Quick reservation, navigation refinements, personal reservation management, demonstration data.
6. Employee onboarding: Admin employee management, Resend invitations, activation tokens, password setup, account activation.
7. Release preparation: Documentation, automated checks, deployment configuration, and demonstration video.

12. Testing and Acceptance Criteria

Area	Acceptance criterion
Authentication	Active users can sign in; inactive/invited accounts cannot sign in before activation.
Authorization	Employees cannot access administrative or unauthorized approval actions.
Resource discovery	Eligible resources and their relevant details display correctly.
Reservation	Valid requests save successfully; invalid intervals and conflicting confirmed bookings are rejected.
Approval	Authorized reviewers can approve/reject pending requests; decisions update status and audit fields.
Personal bookings	Users can view their own reservations and perform permitted actions.
Administration	Administrators can manage resources and invite employees.
Activation	Email invitation leads to successful password setup; token expires and cannot be reused.
UI	Core screens remain usable at common desktop and mobile widths.
Build	TypeScript, lint, and production build checks complete successfully before release.

Manual validation completed during development: An employee invitation email was delivered, the activation link and password setup worked, the newly activated account could sign in, and the temporary personal test account was subsequently removed. Other acceptance criteria should be verified against the deployed environment before declaring production readiness.